

SCALABLE, TRANSFERABLE META-NETWORK FOR DATA SELECTION REQUIRES A DIFFERENT LOSS (AND WHY THE OBVIOUS CHOICE IS PROBLEMATIC)

Zilin Du

College of Computing and Data Science
Nanyang Technological University
Singapore
zilin003@e.ntu.edu.sg

Bowen Yang

College of Computing and Data Science
Nanyang Technological University
Singapore
yang1117@e.ntu.edu.sg

Boyang Albert Li *

College of Computing and Data Science
Nanyang Technological University
Singapore
boyang.li@ntu.edu.sg

ABSTRACT

Data selection is critical for training large language models on massive and heterogeneous corpora. Meta-learning for Training-data Selection offers a principled alternative to heuristic scoring by learning data weights from a target validation objective, but existing methods face a trade-off between fine-grained valuation and transferability to unseen data. A natural solution is to replace per-sample weights with a selection network. However, we find that directly incorporating such a network into existing MTS objectives leads to unstable optimization and poor generalization, caused by weight suppression and persistent reliance on easy-to-learn features. To address these issues, we propose Transferable Example Scoring and Selection (TESS), a scalable data-selection framework built on a Pointwise Value Matching objective (PVM). Experiments on LLM safety and targeted instruction tuning demonstrate strong transfer across datasets, from subsets to full corpora, and from smaller to larger models.

1 INTRODUCTION

The performance of modern AI systems is fundamentally shaped by the training data (Zha et al., 2025; Kumar et al., 2024; Zha et al., 2023; Whang et al., 2023). For large language models (LLMs), scaling requires assembling massive corpora from heterogeneous sources (Zhou et al., 2025; Weber et al., 2024; Langlais et al., 2026; Li et al., 2024a), which often introduces redundant and low-quality examples. Training data selection is hence an important component of LLM training. Many existing methods rank training examples by manually designed scores, such as entropy (Li et al., 2026), instruction-following difficulty (Li et al., 2024b), reducible training loss (Mindermann et al., 2022), and distance to target domain (Deng et al., 2026; Min et al., 2026), to name a few. These scoring rules lack the ability to dynamically adapt to new scenarios. Though each captures a potentially useful signal, it remains an open question how these scores may be combined to create more powerful ranking mechanisms.

Meta-learning for Training-data Selection (MTS) (Shen et al., 2025a; Pan et al., 2025; Ren et al., 2018; Wang et al., 2020; Du et al., 2026; Shu et al., 2019; Yu et al., 2025; Calian et al., 2026) provides a principled alternative, where the values of training data are learned directly from their effects on the validation loss $\mathcal{L}_{\text{val}}$. Let θ denote the LLM parameters, $w_i > 0$ be the learnable weight for the loss of the i -th training data point, $\ell_i(\theta)$, and the vector $\mathbf{w} = (w_i)_i$ contain all w_i as

*Corresponding author.

components. MTS formulates data weighting as the following bilevel optimization problem:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}_{\text{val}}(\boldsymbol{\theta}^*(\mathbf{w})), \quad \text{s. t.} \quad \boldsymbol{\theta}^*(\mathbf{w}) = \arg \min_{\boldsymbol{\theta}} \|\mathbf{w}\|_1^{-1} \sum_i w_i \ell_i(\boldsymbol{\theta}). \quad (1)$$

This framework allows automatic identification of effective training data, which lead the model to generalize to the validation data.

Nevertheless, MTS in practice runs into two obstacles; the first is scalability. Computing the exact hypergradient $\nabla_{\mathbf{w}} \mathcal{L}_{\text{val}}(\boldsymbol{\theta}^*(\mathbf{w}))$ by unrolling the inner optimization is costly. Even unrolling one step and taking the finite-different approximation require three backward operations on $\boldsymbol{\theta}$ for one gradient step on $\mathbf{w}$ (Du et al., 2026; Shu et al., 2019), which is prohibitive expensive for large LLMs. Calian et al. (2026) manage to unroll 8 steps using 64 v5 TPUs, which is beyond the capability of most academic labs. With penalty-based reformulations (Shen & Chen, 2023; Xiao et al., 2023; Shen et al., 2025b), ScaleBiO (Pan et al., 2025) reduces the number of backward calls to two, whereas SEAL (Shen et al., 2025a) reduces it further to one, but incurs discernible performance losses.

The second challenge is generalization to unseen data. In the formulation of Ren et al. (2018); Shen et al. (2025a), every data point receives a separate weight w_i , and no generalization is possible. An alternative approach (Pan et al., 2025) is to assign a separate weight for each data domain (e.g., one dataset in a collection of multiple datasets), which provides generalization within each domain. However, this overlooks heterogeneity among data points in a single domain and cannot generalize to new data domains.

One intuitively appealing idea is to train a meta-network that outputs w_i for every data point. If the meta-network can generalize to unseen data points, we would be able to train on a small dataset and perform data selection on a much larger dataset, which would also substantially improve efficiency and scalability. Though the meta-network approach has been used by Shu et al. (2019); Du et al. (2026); Calian et al. (2026), no such generalization has been reported, not even for the extraordinarily large-scale experiments of Calian et al. (2026). Perhaps unsurprisingly, our experiments find simplistic training of the meta-network leads to unstable optimization (Figure 2) and poor generalization (Table 1 and 3).

Given the massive success of deep learning, the failure of the meta-network to generalize to unseen data is especially perplexing. Our analysis traces the failure to two properties of the training loss. First, as training progresses, most predicted data weights are suppressed toward zero. As a result, over time, the meta-network loses the ability to distinguish useful training data from harmful training data. Second, the loss causes the network to exclusively learn easy features and ignores hard feature, so the network becomes susceptible to shortcut learning.

To improve the scalability and generalization of MTS, we propose Transferable Example Scoring and Selection (TESS). At its core, TESS simplifies ScaleBiO (Pan et al., 2025) and introduces a simple mean squared error objective, which we call Pointwise Value Matching (PVM). TESS constructs sample-level pseudo-labels that capture the utility of training data points and uses these values as supervision for the data-selecting meta-network. Compared to the existing loss, the PVM loss is less focused on easy features, which curbs shortcut learning and promotes generalization. Further, TESS removes the requirement of ScaleBiO to simultaneously train two LLMs with the meta-network, cutting peak memory footprint by more than 20%. Experiments on LLM safety and targeted instruction tuning demonstrate substantially improved generalization of TESS across three settings: cross-dataset transfer, small-to-large model transfer, and subset-to-full-corpus transfer. Our contributions can be summarized as:

- We identify and theoretically analyze two limitations of applying the existing MTS training objective to a shared selection network: competitive weight suppression that undermines the identification of high-quality training data, and persistent preference for easy-to-learn features that can encourage shortcut learning.
- We propose TESS, a transferable data-selection framework that constructs sample-level pseudo-labels and uses them as supervision to learn a data selection network generalizable to unseen data. To the best of our knowledge, this is the first work that demonstrates generalization to unseen data in MTS.

2 RELATED WORK

Meta-Learning Data Selection for LLMs. Scaling meta-learning data selection to LLMs requires efficiency in face of both huge parameter counts (Pan et al., 2025) and dataset sizes (Xie et al., 2023b). Traditional data selection methods (Ren et al., 2018; Shu et al., 2019) differentiate through model updates to compute the hypergradient, which incurs large time and space complexity for large models. ScaleBiO (Pan et al., 2025) and SEAL (Shen et al., 2025a) focus on scaling to model parameter counts. If data selection can be trained on a small dataset and generalize to unseen data, it would also significantly improve scalability to large datasets. However, such generalization remains a difficult challenge.

In terms of generalization, existing approaches can be broadly grouped into sample-level, domain-level, and meta-network methods. Example-level methods (Xie et al., 2023b; Xia et al., 2024; Shen et al., 2025a; Calian et al., 2026) learn one weight for each training data point, thereby offering no generalization to new data points. Domain-level methods (Xie et al., 2023a; 2025; Pan et al., 2025; Yang et al., 2026; Wen et al., 2026) partition the training data into domains and assign one data weight to each domain. Chameleon (Xie et al., 2025) and Data Mixing Agent (Yang et al., 2026) further introduce a shared domain taxonomy that enables limited weight sharing among data domains. However, domain-level weights are too coarse to capture the crucial differences in data quality and content within each domain (Lee et al., 2024). Meta-network methods (Shu et al., 2019; Du et al., 2026; Calian et al., 2026) train a meta-network to output the weight for each data point directly. In theory, this should allow generalization to new data points; in practice, no generalization has been reported. Notably, DataRater (Calian et al., 2026) trains a separate scorer for each dataset, suggesting generalization across dataset is challenging even when the network are trained with a huge amount of data. With this paper, we analyze the reasons behind the challenge mathematically and offer a simple, practical, and high-performance solution.

Shortcut Learning. Shortcut learning occurs when models rely on easy features that are strongly predictive of training labels but whose correlations weaken or reverse under distribution shifts, impairing out-of-distribution generalization (Geirhos et al., 2020; Shah et al., 2020). Prior work distinguishes *predictivity*, how reliably a feature predicts training labels, from *availability*, how easily it can be extracted (Hermann et al., 2024). Scimeca et al. (2021) find that solutions favoring easy-to-learn cues tend to lie in flatter minima and are more abundant in parameter space, while Qiu et al. (2024) show that simpler or more strongly correlated spurious features can slow core-feature learning. We investigate these dynamics in data selection networks. We show that the ScaleBiO loss continues to reinforce easy-to-learn features that are susceptible to shortcut learning, while our proposed PVM promotes the learning of harder and more transferable features.

3 METHODOLOGY

3.1 PROBLEM FORMULATION

Let $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{val}}$ denote the training and target validation sets, with sizes N_{train} and N_{val} , respectively. We further use $\mathcal{D}_{\text{pool}}$ to denote the candidate pool to be scored. For example, $\mathcal{D}_{\text{pool}}$ can be a larger corpus containing $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{train}} \subsetneq \mathcal{D}_{\text{pool}}$, or a disjoint dataset, $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{pool}} = \emptyset$. Let $\ell_i(\theta)$ denote the token-averaged cross-entropy loss for the i -th data point under an LLM parameterized by θ . For the training examples, let $w_i > 0$ denote the weight of the i -th data point, $\mathbf{w} = (w_i)_{i=1}^{N_{\text{train}}}$ the weight vector, and $S = \|\mathbf{w}\|_1$ the total weight.

We aim to learn a data selection meta-network s_ϕ that predicts sample weights and can score unseen examples in $\mathcal{D}_{\text{pool}}$. We parameterize $w_i = s_\phi(x_i) > 0$ and denote the resulting weight vector by $\mathbf{w}(\phi)$. In MTS, learning ϕ is formulated as bilevel optimization:

$$\phi^* = \arg \min_{\phi} \mathcal{L}(\theta^*; \mathcal{D}_{\text{val}}) = \arg \min_{\phi} N_{\text{val}}^{-1} \sum_{i=1}^{N_{\text{val}}} \ell_i(\theta^*), \quad (2)$$

$$\text{s.t. } \theta^* = \arg \min_{\theta} \mathcal{L}(\theta, \mathbf{w}(\phi); \mathcal{D}_{\text{train}}) = \arg \min_{\theta} \|\mathbf{w}\|_1^{-1} \sum_{i=1}^{N_{\text{train}}} w_i \ell_i(\theta). \quad (3)$$

Here, $\mathcal{L}(\theta; \mathcal{D})$ denotes the average per-sample loss, while $\mathcal{L}(\theta, \mathbf{w}(\phi); \mathcal{D})$ denotes the loss weighted by the meta-network ϕ . Under this formulation, we optimize ϕ such that training the LLM with its predicted weights minimizes the loss on $\mathcal{D}_{\text{val}}$.

Algorithm 1 Overview of the TESS Framework**Require:** Training set $\mathcal{D}_{\text{train}}$, validation set $\mathcal{D}_{\text{val}}$, candidate pool $\mathcal{D}_{\text{pool}}$, coefficient α

- 1: Obtain θ_U^* by minimizing $\mathcal{L}(\theta_U; \mathcal{D}_{\text{train}})$
- 2: Obtain θ_W^* by minimizing $\mathcal{L}(\theta_W; \mathcal{D}_{\text{val}}) + \alpha \mathcal{L}(\theta_W; \mathcal{D}_{\text{train}})$
- 3: For each $x_i \in \mathcal{D}_{\text{train}}$, compute the pseudo-label $\Delta_i(\theta_U^*, \theta_W^*) \leftarrow \ell(\theta_U^*; x_i) - \ell(\theta_W^*; x_i)$
- 4: Train the selection network by optimizing $\phi^* \leftarrow \arg \min_{\phi} \mathcal{L}_{\phi}^{\text{PVM}}(\phi; \theta_W^*, \theta_U^*)$ (Eq.6)
- 5: For each $x_j \in \mathcal{D}_{\text{train}}$, predict $\hat{w}_j \leftarrow s_{\phi^*}(x_j)$
- 6: **return** Predicted data values $\hat{\mathbf{w}}_{\text{pool}} = \{\hat{w}_j\}_{x_j \in \mathcal{D}_{\text{pool}}}$

3.2 PRELIMINARIES ON SCALEBIO

ScaleBiO (Pan et al., 2025) provides an efficient framework for MTS at the LLM scale, optimizing data weights w for each data point. Let θ_W denote the LLM parameters and θ_U denote the parameters of an auxiliary LLM. Optimizing θ_U on the training set yields $\theta_U^* = \arg \min_{\theta_U} \mathcal{L}(\theta_U, w; \mathcal{D}_{\text{train}})$. The original constraint in the bi-level problem (Eq. 1) can then be rewritten as $\theta_W = \theta_U^*$.

Relaxing the constraint to a necessary condition that θ_W and θ_U^* have the same training loss, we obtain a new constraint: $\mathcal{L}(\theta_W, w; \mathcal{D}_{\text{train}}) - \min_{\theta_U} \mathcal{L}(\theta_U, w; \mathcal{D}_{\text{train}}) = 0$. Building the Lagrangian function with the penalty coefficient $\alpha > 0$, we derive a new minimax problem:

$$\min_{w, \theta_W} \max_{\theta_U} \mathcal{L}(\theta_W; \mathcal{D}_{\text{val}}) + \alpha \left[\mathcal{L}(\theta_W, w; \mathcal{D}_{\text{train}}) - \mathcal{L}(\theta_U, w; \mathcal{D}_{\text{train}}) \right]. \quad (4)$$

To solve Eq.4, ScaleBiO simultaneously trains the two LLMs and data weights, resulting in substantial memory overhead. SEAL (Shen et al., 2025a) additionally eliminates the auxiliary LLM θ_U , but this comes at an accuracy penalty.

We write the ScaleBiO loss for data weights w explicitly:

$$\mathcal{L}_w^{\text{SBO}}(w; \theta_W, \theta_U) = \|w\|_1^{-1} \sum_{i=1}^{N_{\text{train}}} w_i [\ell_i(\theta_W) - \ell_i(\theta_U)]. \quad (5)$$

It is evident that, to optimize $\mathcal{L}_w^{\text{SBO}}$, we should assign high w_i to data points with a low loss on θ_w and a high loss on θ_U . Note that θ_w is trained on both the training set $\mathcal{D}_{\text{train}}$ and the validation set $\mathcal{D}_{\text{val}}$ but θ_U is trained on $\mathcal{D}_{\text{train}}$ only. Therefore, ScaleBiO selects data points that benefit from additional training on $\mathcal{D}_{\text{val}}$, which are likely more similar to $\mathcal{D}_{\text{val}}$ than $\mathcal{D}_{\text{train}}$.

3.3 TRANSFERABLE EXAMPLE SCORING AND SELECTION (TESS)

Incorporating a Selection Network. We extend ScaleBiO by replacing the independent data weights w_i with the data selection meta-network $s_{\phi}(x_i)$. This yields the ScaleBiO meta-network objective (SBO):

$$\mathcal{L}_{\phi}^{\text{SBO}}(\phi; \theta_W, \theta_U) = \left(\sum_{i=1}^{N_{\text{train}}} s_{\phi}(x_i) \right)^{-1} \sum_{i=1}^{N_{\text{train}}} s_{\phi}(x_i) [\ell_i(\theta_W) - \ell_i(\theta_U)]. \quad (6)$$

However, empirically optimizing the meta-network with $\mathcal{L}_{\phi}^{\text{SBO}}$ leads to unstable optimization (Figure 2) and poor generalization (Tables 1 and 3). We find this can be simply addressed by Pointwise Value Matching, presented below.

Pointwise Value Matching. The key idea is to use the loss difference between the two LLMs as a proxy for sample utility. Recall that θ_U is trained only on $\mathcal{D}_{\text{train}}$, whereas θ_W is additionally guided by $\mathcal{D}_{\text{val}}$. For each training example x_i , we define $\Delta_i(\theta_U, \theta_W) := \ell_i(\theta_U) - \ell_i(\theta_W)$. A larger Δ_i indicates a greater loss reduction under the validation-guided model relative to the training-only model, suggesting greater relevance to the target validation objective. We use this difference as a pseudo-label and train the meta-network to match it:

$$\mathcal{L}_{\phi}^{\text{PVM}}(\phi; \theta_W, \theta_U) = N_{\text{train}}^{-1} \sum_{i=1}^{N_{\text{train}}} (s_{\phi}(x_i) - \Delta_i(\theta_U, \theta_W))^2. \quad (7)$$

Sequential Training of TESS. Unlike ScaleBiO, TESS separates the training of the three models. Specifically, using uniform sample weights, we obtain θ_U^* by minimizing $\mathcal{L}(\theta_U; \mathcal{D}_{\text{train}})$ and θ_W^* by minimizing $\mathcal{L}(\theta_W; \mathcal{D}_{\text{val}}) + \alpha \mathcal{L}(\theta_W; \mathcal{D}_{\text{train}})$. We then freeze both models, compute $\Delta_i(\theta_U^*, \theta_W^*)$ for

each training example, and train the meta-network by minimizing $\mathcal{L}_\phi^{\text{PVM}}(\phi; \theta_W^*, \theta_U^*)$. This sequential design has two advantages over ScaleBiO. First, it avoids simultaneously optimizing two LLMs and the selection network, reducing peak memory consumption. Second, it allows the pretrained LLMs and the randomly initialized meta-network to use independent optimization schedules. The trained meta-network directly scores unseen examples in $\mathcal{D}_{\text{pool}}$ without further optimization. The complete procedure of TESS is summarized in Algorithm 1.

4 WHY THE SCALEBIO LOSS PERFORMS POORLY

We next examine why optimizing with the ScaleBiO objective can fail to produce a transferable meta-network. We identify two properties of the ScaleBiO objective: competitive weight suppression, which weakens sample discrimination, and a persistent preference for easy-to-learn features, which can encourage shortcut learning.

4.1 DATA WEIGHT SUPPRESSION AND LOSS OF DISCRIMINATION

Our first observation is that the ScaleBiO objective drives most predicted weights toward zero, alongside losing the power of identifying good data (Figure 2). This occurs under both joint training in ScaleBiO and sequential training in TESS when training with $\mathcal{L}_\phi^{\text{SBO}}$. Our analysis below strictly applies to sequential training in TESS, where the two LLMs are fixed. Empirically, we observe qualitatively similar dynamics under the joint training in ScaleBiO.

We first treat the weights of training examples as independent hyperparameters. For the selection network, $w_i = s_\phi(x_i)$ is updated through shared parameters ϕ . However, the update directions on w_i with and without ϕ are similar. With $\Delta_i^* := \Delta_i(\theta_U^*, \theta_W^*)$ and $N = N_{\text{train}}$, the ScaleBiO objective and its gradient with respect to each data weight are

$$\mathcal{L}_\phi^{\text{SBO}} = - \sum_{i=1}^N p_i \Delta_i^*, \quad \frac{\partial \mathcal{L}_\phi^{\text{SBO}}}{\partial w_i} = - \frac{\Delta_i^* - \bar{\Delta}^*}{S}, \quad (8)$$

where $S = \sum_{j=1}^N w_j$ is the total weight, $p_i = w_i/S$ the normalized weight, and $\bar{\Delta}^* = \sum_i p_i \Delta_i^*$ the weighted-average pseudo-label. The derivation is provided in Appendix A.

The gradient reveals a competitive mechanism: examples are up-weighted when $\Delta_i^* > \bar{\Delta}^*(t)$ and down-weighted otherwise. As examples with larger pseudo-labels gain relative weight, the weighted mean rises, making it harder for other examples to receive positive updates. This can produce a *winner-takes-most* allocation, in which a single weight dominates the total mass. Given a lower bound $\delta_0 \geq 0$ of data weight, the following theorem is given by Du et al. (2026) (proof reproduced in Appendix B).

Theorem 1 (Concentration of normalized data weights p). *Assume there is a unique index K such that $\Delta_K^* > \Delta_i^*$ for all $i \neq K$, then $\lim_{t \rightarrow \infty} p_K(t) = 1$ and $\lim_{t \rightarrow \infty} p_i(t) = 0$. For $\delta_0 = 0$, this limit is reached in finite time. For $\delta_0 > 0$, $1 - p_K(t) = O(t^{-1/3})$.*

Different from Du et al. (2026), who use weight concentration to analyze gradient signal-to-noise ratios, we examine its implications for sample discrimination. The ScaleBiO objective eventually allocates nearly all weight to one training data point. At the beginning, the meta-network can differentiate high-quality data points from low-quality data points by assigning them different weights. As training progresses, however, its outputs become less discriminative. This creates a strange phenomenon that more training leads to worsened data selection even on the training set itself.

Empirical verification. Figure 1 illustrates this behavior through a numerical simulation. The weight w_1 , associated with the largest pseudo-label, becomes dominant, while the remaining weights reach the lower bound. Notably, w_4 initially increases but later decreases. Between $t = 0.4$ and $t = 0.6$, the weights can separate good data

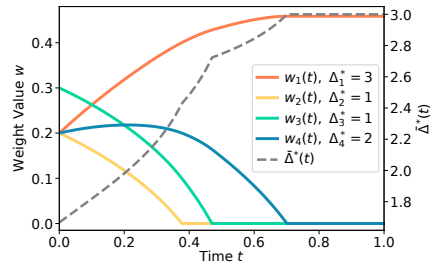


Figure 1: Simulation of the weight dynamics in Theorem 1. Curves show the weights $w_i(t)$, with fixed pseudo-labels Δ_i^* indicated in the legend. Initial weights are $(0.2, 0.2, 0.3, 0.2)$, and δ_0 is 10^{-6} .

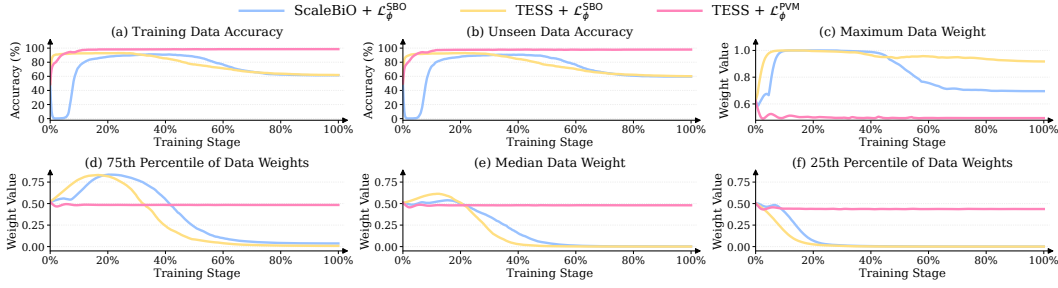


Figure 2: Training dynamics of ScaleBiO and TESS under different meta-network objectives. With SBO loss, most predicted weights shrink toward zero as accuracy declines. In contrast, PVM loss maintains nonvanishing weight quantiles and high accuracy.

with the highest pseudo-labels, but this period is transient. Some data selection can be achieved by carefully tuning early stopping (consistent with Figure 9 of Calian et al. (2026)) but it requires significant trial and error.

To verify these dynamics, we construct a controlled dataset by mixing GSM8K and unsafe examples at a 2:1 ratio, together with a disjoint unseen set of the same composition. This way, we know exactly what data points are unsafe and should be discarded. We rank examples by predicted weights and evaluate how accurately the bottom one-third identifies unsafe examples on both seen and unseen data. Experimental details are provided in Appendix E. Figure 2 compares ScaleBiO with $\mathcal{L}_\phi^{\text{SBO}}$, TESS with $\mathcal{L}_\phi^{\text{SBO}}$, and TESS with $\mathcal{L}_\phi^{\text{PVM}}$.

Under $\mathcal{L}_\phi^{\text{SBO}}$, the 25th percentile, median, and 75th percentile of the predicted weights progressively approach zero, consistent with the competitive suppression analyzed above. The maximum weight initially increases but later decreases. This is expected because a shared selection network typically produces smoother outputs than a one-hot vector (Ma & Ying, 2021), causing even dominant weights to decrease as most outputs are suppressed.

Weight suppression accompanies declining discrimination. For ScaleBiO, seen and unseen accuracy first improve but eventually fall to around 60%. TESS with the same $\mathcal{L}_\phi^{\text{SBO}}$ exhibits a similar long-term decline, although it improves immediately at the beginning of training. We attribute this early difference to the quality of the supervision signal: ScaleBiO updates the meta-network while the two LLMs are still being optimized, whereas TESS computes loss gaps from well-trained, fixed LLMs. In contrast, TESS with $\mathcal{L}_\phi^{\text{PVM}}$ maintains stable, nonvanishing weight quantiles and reaches 98.58% seen accuracy and 97.96% unseen accuracy.

4.2 EASY-FEATURE DOMINANCE AND SHORTCUT LEARNING

The previous subsection links weight suppression to reduced sample discrimination. We now examine a second limitation: ScaleBiO can continue favoring easy-to-learn features after they have been substantially fitted, whereas PVM can shift learning toward harder features that explain the remaining prediction error. This distinction matters for generalization when an easy feature is predictive on the training data but unreliable on unseen data, thereby acting as a shortcut (Hermann et al., 2024; Geirhos et al., 2020).

Denote the empirical covariance between random variables u and v as $\widehat{\text{Cov}}_N(u, v) := N^{-1} \sum_{i=1}^N (u_i - \bar{u})(v_i - \bar{v})$. Using fixed pseudo-labels $\Delta_i = \Delta_i(\theta_U^*, \theta_W^*)$, for two data features e and h , we denote their covariance with pseudo-labels as

$$c_e = \widehat{\text{Cov}}_N(\Delta, e), \quad c_h = \widehat{\text{Cov}}_N(\Delta, h). \quad (9)$$

We call e the *easy feature* and h the *hard feature* if $|c_e| > |c_h|$. For each training example x_i , let e_i and h_i be its easy and hard features, both standardized to zero mean and unit standard deviation. For simplicity, we analyze a one-layer meta-network with parameters $\phi = (\beta_0, \beta_e, \beta_h)$ to predict weights for x_i . It predicts $w_i = s_\phi(x_i) = a_i$, where $a_i = \beta_0 + \beta_e e_i + \beta_h h_i$ is the output logit.

To examine the learning preference for features, we compare the gap between gradients of easy and hard features in PVM and ScaleBiO training.

Table 1: Safe–unsafe data classification accuracy under different shortcut strengths. Cov and ρ denote the empirical covariance and Pearson correlation coefficient between the shortcut feature and the label, respectively. Early, middle, and late stages correspond to 20%, 50%, and 80% of meta-network training, respectively. Acc_S and Acc_U denote seen and unseen accuracy, and $\text{Gap} = \text{Acc}_S - \text{Acc}_U$ denotes the generalization gap.

Shortcut (Cov / ρ)	Method	Early Stage			Middle Stage			Late Stage			Avg. Gap
		Acc_S	Acc_U	Gap	Acc_S	Acc_U	Gap	Acc_S	Acc_U	Gap	
Strong 0.17 / 0.57	TESS + $\mathcal{L}_\phi^{\text{PVM}}$	98.84	79.38	<u>19.46</u>	99.56	84.62	14.94	99.56	84.97	14.59	16.33
	TESS + $\mathcal{L}_\phi^{\text{SBO}}$	<u>97.96</u>	72.44	25.52	90.22	52.26	37.96	<u>84.71</u>	<u>46.31</u>	<u>38.40</u>	33.96
	ScaleBiO + $\mathcal{L}_\phi^{\text{SBO}}$	93.42	<u>78.84</u>	14.58	<u>90.76</u>	<u>57.73</u>	<u>33.03</u>	76.00	32.44	43.56	<u>30.39</u>
Weak 0.06 / 0.41	TESS + $\mathcal{L}_\phi^{\text{PVM}}$	98.04	91.47	6.57	98.67	93.51	5.16	98.76	93.60	5.16	5.63
	TESS + $\mathcal{L}_\phi^{\text{SBO}}$	<u>96.53</u>	80.71	15.82	87.82	65.78	22.04	<u>84.00</u>	<u>60.53</u>	23.47	20.44
	ScaleBiO + $\mathcal{L}_\phi^{\text{SBO}}$	91.64	<u>82.40</u>	<u>9.24</u>	<u>90.13</u>	<u>71.73</u>	<u>18.40</u>	75.56	55.56	<u>20.00</u>	<u>15.88</u>

Theorem 2 (Contrasting gradient-gap dynamics). *At training step t , denote the feature gradients by $G_k^q(\phi_t) := \partial \mathcal{L}_{\phi_t}^q / \partial \beta_{k,t}$, where $k \in \{e, h\}$ and $q \in \{\text{SBO}, \text{PVM}\}$. Define gradient gap $D_t^q = (G_e^q(\phi_t))^2 - (G_h^q(\phi_t))^2$, then D_t^{PVM} decreases exponentially with t while D_t^{SBO} remains non-decreasing. Specifically, when initializing $\beta_e = \beta_h = 0$ and $\beta_0 > 0$, there exists a training step $T > 0$ and $\lambda \in (0, 1)$, such that $\forall t < T$,*

$$D_t^{\text{PVM}} = (1 - \lambda)^t D_0^{\text{PVM}}, \quad D_t^{\text{SBO}} \geq D_{t-1}^{\text{SBO}}. \quad (10)$$

The proof is provided in Appendix C. This comparison explains how PVM can gradually balance the information learned from easy and hard features, while ScaleBiO continues to prefer easy features. This links the training objective to shortcut learning. When the easy feature is predictive on the training data but unreliable on unseen data, ScaleBiO’s persistent preference can reinforce reliance on this shortcut. In contrast, PVM mitigates the gradient gap between the easy feature and the hard feature. This provides a mechanism for TESS to move beyond training-specific shortcuts and improve data selection on unseen examples.

Empirical verification. To test shortcut reliance, we add shortcut features to the seen training data and flip them on unseen data (Appendix E). We use the gap between seen and unseen dataset accuracy as a proxy for generalization, with smaller gaps indicating better generalization on unseen data. Table 1 shows clear differences in training dynamics. Under both shortcut strengths, TESS with $\mathcal{L}_\phi^{\text{PVM}}$ improves unseen accuracy and reduces the gap from early to late training. In contrast, $\mathcal{L}_\phi^{\text{SBO}}$ leads to lower unseen accuracy and larger gaps over training in both frameworks. These results are consistent with our analysis that SBO persistently learning more shortcut features, and support PVM as a more effective objective for learning transferable meta-networks. Moreover, all variants exhibit larger average gaps under strong shortcuts, indicating that stronger spurious correlations make generalization more difficult.

5 EXPERIMENTS

5.1 LLM SAFETY

We evaluate whether a meta-network can identify training examples that increase harmful behavior finetuning a target LLM. We use Alpaca (Taori et al., 2023; Peng et al., 2023) and Dolly (Conover et al., 2023) as candidate pools $\mathcal{D}_{\text{pool}}$. The validation set from He et al. (2024) contains 100 harmful instructions paired with safe refusal responses. We finetune the target LLM on 1,000 lowest-scoring examples and evaluate it on DirectHarm4 (DH4) (Lyu et al., 2024), HarmBench (HB) (Mazeika et al., 2024), and HEx-PHI (HEx) (Qi et al., 2024). We use LlamaGuard 3 (Llama Team, 2024) as the classifier to compute attack success rate (ASR) from generated responses; higher ASR indicates more effective harmful-data selection. The target models are Llama-3-8B-Instruct, Llama-3.1-8B-Instruct (Llama Team, 2024), and Qwen2.5-7B-Instruct (Qwen et al., 2025). Implementation details are provided in Appendix D.

Settings and Baselines. We consider two settings. In *Training*, the meta-network is trained and applied on the same candidate dataset, with $\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{pool}}$. In *Generalization*, it is trained on Alpaca and directly applied to Dolly, or vice versa, with $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{pool}} = \emptyset$. We compare against

Table 2: Attack success rate (ASR, %; higher is better) for harmful-data selection on Alpaca and Dolly. Best and second-best results are **bold** and underlined, ranked separately within two settings.

Setting		Training								Generalization		
Dataset	Bench	Random	Task-agnostic Heuristics		Learnable Weighting							
			GradSafe	Bi-Anchor	SEAL _w	SBO _w	SEAL _φ	SBO _φ	TESS	SEAL _φ	SBO _φ	TESS
Target LLM: Llama3-8B-Instruct												
Alpaca	DH4	25.00	28.00	49.00	26.75	<u>38.25</u>	12.50	12.50	36.75	<u>8.50</u>	6.50	35.50
	HB	15.00	16.00	35.00	13.50	<u>21.00</u>	9.00	7.00	20.50	<u>6.00</u>	4.00	25.00
	HEX	6.55	8.97	24.58	6.90	<u>10.69</u>	5.86	3.44	<u>11.38</u>	3.10	<u>3.44</u>	15.52
Dolly	DH4	55.25	<u>75.00</u>	74.50	60.75	69.25	73.00	74.50	82.25	<u>56.25</u>	43.00	71.25
	HB	39.25	<u>82.00</u>	75.00	59.50	73.00	71.25	69.50	85.50	<u>43.50</u>	27.50	59.50
	HEX	31.38	<u>74.14</u>	67.59	42.76	53.44	61.03	63.44	81.72	<u>30.69</u>	22.07	52.41
Average		28.74	47.35	54.28	35.03	44.27	38.77	38.40	<u>53.02</u>	<u>24.67</u>	17.75	43.20
Target LLM: Llama3.1-8B-Instruct												
Alpaca	DH4	22.50	7.50	11.00	<u>27.75</u>	20.50	8.50	5.50	40.25	2.25	<u>11.00</u>	34.25
	HB	18.50	5.00	12.50	<u>13.00</u>	8.00	7.50	3.50	18.50	2.50	<u>7.00</u>	16.50
	HEX	<u>8.97</u>	3.45	3.10	6.90	2.41	2.76	1.03	12.41	1.03	<u>4.83</u>	9.66
Dolly	DH4	54.00	59.50	67.25	71.75	63.75	64.00	<u>74.50</u>	81.00	<u>71.50</u>	60.25	80.25
	HB	51.00	60.50	50.50	<u>65.00</u>	58.50	55.00	<u>60.00</u>	80.00	<u>69.00</u>	50.50	81.00
	HEX	29.30	33.79	40.00	38.62	33.10	33.45	<u>42.07</u>	60.34	<u>40.69</u>	28.28	71.38
Average		30.71	28.29	30.73	<u>37.17</u>	31.04	28.54	31.10	48.75	<u>31.16</u>	26.98	48.84
Target LLM: Qwen2.5-7B-Instruct												
Alpaca	DH4	27.50	26.00	44.50	20.00	27.00	<u>28.25</u>	11.75	44.50	8.75	<u>11.00</u>	38.75
	HB	11.00	10.00	24.50	9.00	8.00	17.50	3.50	<u>23.50</u>	<u>3.00</u>	2.50	16.00
	HEX	13.10	12.07	<u>24.80</u>	6.55	8.28	9.31	6.56	24.83	<u>7.24</u>	6.56	18.28
Dolly	DH4	50.50	<u>66.50</u>	60.50	49.75	65.75	60.25	59.75	86.50	48.00	<u>62.25</u>	84.50
	HB	36.00	<u>60.00</u>	60.50	<u>65.50</u>	61.50	43.00	45.50	87.00	34.00	<u>50.50</u>	83.50
	HEX	32.41	<u>51.03</u>	42.07	<u>51.03</u>	50.34	45.86	44.48	88.62	27.59	<u>29.66</u>	80.00
Average		28.42	37.60	<u>42.81</u>	33.64	36.81	34.03	28.59	59.16	21.43	<u>27.08</u>	53.51
Overall Average		29.29	37.75	<u>42.61</u>	35.28	37.38	33.78	32.70	53.64	<u>25.76</u>	23.94	48.51

the random selection baseline and two groups of task-agnostic methods. GradSafe (Xie et al., 2024) and Bi-Anchoring (Bi-Anchor) (He et al., 2024) are heuristic methods that rank examples using distances relative to the validation set. SEAL (Shen et al., 2025a) and ScaleBiO (Pan et al., 2025) are meta-learning methods that learn data weights. We create two variants for each. SEAL_w and SBO_w optimize one weight per example, whereas SEAL_φ and SBO_φ train a meta-network $s_φ$ to predict weights.

Main Results. Table 2 shows the results. Under *Training* settings, TESS achieves the strongest performance, exceeding the strongest learning baseline SBO_w by 16.27% and the strongest heuristic Bi-Anchor by 11.04%. Although Bi-Anchor remains stronger on Llama3-Alpaca, TESS achieves the best in 14 of the 18 comparisons. These gains stem from the optimization stability of TESS. In contrast, simply replacing individual weights with a shared meta-network does not consistently help: SEAL_φ and SBO_φ underperform their weight-based counterparts by 1.50 and 4.68 points on average. This suggests that the training objective is critical for effective selection.

Under the *Generalization* setting, TESS outperforms SEAL_φ and SBO_φ in all 18 evaluations, surpassing SEAL_φ and SBO_φ by 22.76 and 24.58 points in average ASR, respectively, while dropping only 5.13% from its training performance. These results support our analysis that PVM learns more transferable selection signals rather than relying on training-specific shortcuts.

Small-to-Large Model Transfer. We further evaluate whether a meta-network trained using a smaller LLM can select useful data for a larger model. We train the meta-network with Qwen2.5-0.5B-Instruct and use it to select data for fine-tuning Qwen2.5-7B-Instruct. As shown in Table 5 (Appendix F), this reduces training time by 5.65–6.60× while retaining 77.53% and 83.67% of the corresponding large-model performance under the *Training* and *Generalization* settings. This demonstrates that TESS can transfer selection signals across model scales with substantially lower training cost. This offers a promising direction for scaling data selection to larger models efficiently.

Table 3: Ablation of Different Training Losses for the Meta-network on Qwen2.5-7B-Instruct.

Setting	Method	Alpaca			Dolly			Avg.
		DH4	HB	HEX	DH4	HB	HEX	
Training	TESS w/ PVM Loss	44.50	23.50	24.83	86.50	87.00	88.62	59.16
	TESS w/ SBO Loss	39.50	19.50	22.06	58.25	40.50	45.52	37.56 ^{-21.60}
	TESS w/ KL Loss	<u>40.75</u>	19.00	<u>23.86</u>	60.75	53.50	59.66	<u>42.92</u> ^{-16.24}
	Pseudo-Label Ranking	11.25	7.50	5.86	<u>75.75</u>	<u>75.00</u>	<u>70.69</u>	41.01 ^{-18.15}
Generalization	TESS w/ PVM Loss	38.75	16.00	18.28	84.50	83.50	80.00	53.51
	TESS w/ SBO Loss	<u>12.00</u>	<u>9.50</u>	<u>7.24</u>	59.25	44.00	41.37	<u>28.89</u> ^{-24.61}
	TESS w/ KL Loss	11.75	4.00	6.55	<u>61.00</u>	43.50	<u>43.79</u>	28.43 ^{-25.07}

Table 4: Llama-2-7B performance after fine-tuning on N selected Tulu V2 examples.

Method	GSM8K				CodeX				Overall Avg.
	$N=1K$	$N=5K$	$N=10K$	Avg.	$N=5K$	$N=10K$	$N=20K$	Avg.	
Random	13.41	15.92	16.13	15.15	29.05	<u>27.03</u>	27.70	27.93	21.54
LESS	18.80	<u>22.36</u>	22.59	<u>21.25</u>	25.00	26.35	28.37	26.57	23.91 ^{+2.37}
RDS+	<u>17.43</u>	21.60	<u>23.57</u>	20.87	28.37	26.35	29.73	<u>28.15</u>	<u>24.51</u> ^{+2.97}
SEAL _w	14.78	16.37	17.89	16.35	24.32	25.00	25.68	25.00	20.67 ^{-0.87}
SBO _w	15.01	17.74	18.65	17.13	26.35	25.68	27.03	26.35	21.74 ^{+0.20}
SEAL _φ	17.74	19.56	19.48	18.93	24.32	<u>27.03</u>	26.35	25.90	22.41 ^{+0.87}
SBO _φ	17.43	18.65	19.03	18.37	26.35	25.68	27.70	26.58	22.47 ^{+0.93}
Ours	20.43	25.47	25.70	23.87	29.73	28.37	<u>29.05</u>	29.05	26.46 ^{+4.92}

Ablation Study. We ablate the training losses within the TESS framework on Qwen2.5-7B-Instruct, comparing PVM Loss (*default*), SBO Loss, KL Loss, together with Pseudo-Label Ranking, which directly ranks examples by Δ^* without training a meta-network. KL Loss (Appendix D.5) matches normalized meta-network outputs to a pseudo-label-reweighted target distribution that favors examples with higher estimated utility.

Under the *Training* setting, TESS with PVM Loss achieves the highest ASR across all six dataset-benchmark pairs, outperforming SBO Loss and KL Loss by 21.60% and 16.24%, respectively. These gains stem from the more stable optimization of PVM. Notably, PVM Loss also outperforms *Pseudo-Label Ranking* by 18.15%. One possible explanation is that the meta-network may implicitly denoise pseudo-labels by learning shared patterns (Liu et al., 2020; Li et al., 2020). Under the *Generalization* setting, PVM Loss exceeds SBO Loss and KL Loss by 24.61% and 25.07% on average, respectively. Moreover, its average ASR decreases by only 5.65 points relative to the *Training* setting, compared with drops of 8.67% for SBO Loss and 14.49% for KL Loss. The stronger unseen-data performance and smaller degradation indicate that PVM learns a more transferable meta-network.

5.2 TARGETED INSTRUCTION TUNING

We evaluate if TESS trained on a small subset can generalize to the full instruction tuning corpus. GSM8K (Cobbe et al., 2021) and Codex (Chen et al., 2021) are the target tasks. We train the meta-network using a random 50K-example subset of Tulu V2 (Iverson et al., 2025) and a task-specific validation set, then apply it to select training data from the 197K full Tulu V2. Following Nayak et al. (2026), we finetune Llama-2-7B on the selected examples and report GSM8K accuracy and Codex pass@10. Implementation details are provided in Appendix D. We compare against random selection, heuristic-based methods, and learnable-weighting baselines. The heuristic methods, LESS (Xia et al., 2024) and RDS+ (Iverson et al., 2025), rank candidate examples using predefined measures of relevance to the target validation set. All four learnable baselines are trained on the same 50K subset. SEAL_w and SBO_w are restricted to select from their training set.

Main Results. Table 4 shows that TESS achieves the best average performance on both tasks and ranks first in five of six settings, exceeding Random by 4.92 points overall. TESS improves over heuristic-based selection, exceeding LESS and RDS+ in overall average by 2.55% and 1.95%, respectively. The advantage is more pronounced on GSM8K, where TESS outperforms the strongest heuristic baseline, LESS, in task-average performance by 2.62%. TESS also consistently outperforms all learnable-weighting baselines. Compared with SEAL_φ and SBO_φ, which use the same 50K training subset and full selection pool, TESS improves the overall average by 4.05 and 3.99

points, respectively. This controlled comparison shows that the performance gains come from the new loss rather than merely using a shared meta-network.

Regarding training efficiency, SEAL_w and SBO_w optimize sample-specific weights and thus must be trained on the full 200K pool, whereas TESS trains on only 50K examples and generalizes to the full pool. As a result, SBO_w and SEAL_w require 59.8 and 39.6 hours, respectively, corresponding to $3.46\times$ and $1.95\times$ more training time than the 13.4 hours required by TESS. SBO_w also incurs higher memory usage due to jointly training multiple models, requiring 188.2 GB compared with 156.8 GB for TESS, an increase of 31.4GB (20.03%).

6 CONCLUSION

We introduced TESS, a transferable data-selection framework that learns a selector from sample-level pseudo-labels. We identify weight suppression and easy-feature preference of ScaleBiO objective and address them with PVM. TESS shows strong transfer across datasets, corpus scales, and model sizes.

AI USE STATEMENT

In this work, we used generative AI tools to formulate mathematical claims, assist in the writing of proofs, provide feedback on research methodology or experiments, help develop theoretical models or conceptual frameworks, provide critical ingredients for proving mathematical claims, and assist with translation. We have not used generative AI tools to generate synthetic data sets, implement methods, support qualitative and thematic data analysis, or interpret results, and propose or refine hypotheses and clean and reformat dataset are not applicable to this work.

Additionally, we used generative AI tools to edit software code, search for information, and edit a research paper to improve readability. We have not used generative AI tools to create or modify scientific figures or images, suggest experimental parameters, draft parts of a research paper, summarize or analyse existing literature, discover research topics or identify gaps, brainstorm, identify relevant literature, format references, suggest a structure for a research paper, or propose a title or keywords for a research paper. Formulate questions for surveys or interviews, creation of artifacts, and transcribe recordings of research material are not applicable to this work.

We have reviewed all AI-assisted work. All AI-assisted text, mathematical results, and information-search results were manually verified and revised by the authors as needed. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REPRODUCIBILITY STATEMENT

We provide detailed descriptions of the models, datasets, training configurations, and evaluation protocols in Section 5.1 and Appendix D. All datasets and pretrained models used in this work are from prior work and are publicly available. We will also release our code upon acceptance to facilitate reproducibility.

REFERENCES

- Dan Andrei Calian, Greg Farquhar, Iurii Kemaev, Luisa Zintgraf, Matteo Hessel, Jeremy Shar, Junhyuk Oh, András György, Tom Schaul, Jeff Dean, et al. Datarater: Meta-learned dataset curation. *Advances in Neural Information Processing Systems*, 38:20060–20092, 2026.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- Ruibo Chen, Yihan Wu, Lichang Chen, Guodong Liu, Qi He, Tianyi Xiong, Chenxi Liu, Junfeng Guo, and Heng Huang. Your vision-language model itself is a strong filter: Towards high-quality instruction tuning with data selection. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 4156–4172, 2024. doi: 10.18653/v1/2024.findings-acl.246.

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168. URL <https://arxiv.org/abs/2110.14168>.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free dolly: Introducing the world’s first truly open instruction-tuned llm, 2023. URL <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>.
- Xun Deng, Han Zhong, Rui Ai, Fuli Feng, Zheng Wang, and Xiangnan He. Less is more: Improving llm alignment via preference data selection. *Advances in Neural Information Processing Systems*, 38:161259–161285, 2026.
- Zilin Du, Junqi Zhao, and Boyang Albert Li. On the difficulty of learning a meta-network for training data selection. *arXiv preprint arXiv:2606.00571*, 2026.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.
- Luxi He, Mengzhou Xia, and Peter Henderson. What is in your safe data? identifying benign data that breaks safety. *arXiv preprint arXiv:2404.01099*, 2024.
- Katherine Hermann, Hossein Mobahi, Thomas Fel, and Michael Mozer. On the foundations of shortcut learning. In *International Conference on Learning Representations*, volume 2024, pp. 43832–43868, 2024.
- Hamish Ivison, Muru Zhang, Faeze Brahman, Pang Wei Koh, and Pradeep Dasigi. Large-scale data selection for instruction tuning. *arXiv preprint arXiv:2503.01807*, 2025.
- Jiaming Ji, Mickel Liu, Josef Dai, Xuehai Pan, Chi Zhang, Ce Bian, Boyuan Chen, Ruiyang Sun, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. *Advances in Neural Information Processing Systems*, 36:24678–24704, 2023.
- Sushant Kumar, Sumit Datta, Vishakha Singh, Sanjay Kumar Singh, and Ritesh Sharma. Opportunities and challenges in data-centric ai. *IEEE Access*, 12:33173–33189, 2024.
- Pierre-Carl Langlais, Pavel Chizhov, Catherine Arnett, Carlos Hinostroza, Mattia Nee, Eliot Jones, Irène Girard, David Mach, Anastasia Stasenko, and Ivan Yamshchikov. Common corpus: The largest collection of ethical data for llm pre-training. In *International Conference on Learning Representations*, volume 2026, pp. 25827–25860, 2026.
- Jaewoo Lee, Boyang Li, and Sung Ju Hwang. Concept-skill transferability-based data selection for large vision-language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 5060–5080, 2024. doi: 10.18653/v1/2024.emnlp-main.291.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, et al. Datacomp-llm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024a.
- Ming Li, Yong Zhang, Zhitao Li, Jiuhai Chen, Lichang Chen, Ning Cheng, Jianzong Wang, Tianyi Zhou, and Jing Xiao. From quantity to quality: Boosting llm performance with self-guided data selection for instruction tuning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7602–7635, 2024b.
- Mingchen Li, Mahdi Soltanolkotabi, and Samet Oymak. Gradient descent with early stopping is provably robust to label noise for overparameterized neural networks. In *International conference on artificial intelligence and statistics*, pp. 4313–4324. PMLR, 2020.

- Xiaoyuan Li, Yubo Ma, Chengpeng Li, Fengbin Zhu, Yiyao Yu, Keqin Bao, Wenjie Wang, Fuli Feng, and Dayiheng Liu. Unified data selection for llm reasoning. *arXiv preprint arXiv:2605.22389*, 2026.
- Sheng Liu, Jonathan Niles-Weed, Narges Razavian, and Carlos Fernandez-Granda. Early-learning regularization prevents memorization of noisy labels. *Advances in neural information processing systems*, 33:20331–20342, 2020.
- AI @ Meta Llama Team. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Kaifeng Lyu, Haoyu Zhao, Xinran Gu, Dingli Yu, Anirudh Goyal, and Sanjeev Arora. Keeping llms aligned after fine-tuning: The crucial role of prompt templates. *Advances in Neural Information Processing Systems*, 37:118603–118631, 2024.
- Chao Ma and Lexing Ying. On linear stability of sgd and input-smoothness of neural networks. *Advances in Neural Information Processing Systems*, 34:16805–16817, 2021.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, et al. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024.
- Guanghui Min, Tianhao Huang, Ke Wan, and Chen Chen. Gist: Targeted data selection for instruction tuning via coupled optimization geometry. *arXiv preprint arXiv:2602.18584*, 2026.
- Sören Mindermann, Jan M Brauner, Muhammed T Razzak, Mrinank Sharma, Andreas Kirsch, Winnie Xu, Benedikt Höltingen, Aidan N Gomez, Adrien Morisot, Sebastian Farquhar, et al. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*, pp. 15630–15649. PMLR, 2022.
- Nihal V. Nayak, Paula Rodriguez-Diaz, Neha Hulkund, Sara Beery, and David Alvarez-Melis. A critical look at targeted instruction selection: Disentangling what matters (and what doesn’t). In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.14696>.
- Rui Pan, Xiang Liu, Shizhe Diao, Renjie Pi, Jipeng Zhang, Chi Han, and Tong Zhang. Lisa: Layerwise importance sampling for memory-efficient large language model fine-tuning. *Advances in Neural Information Processing Systems*, 37:57018–57049, 2024.
- Rui Pan, Dylan Zhang, Hanning Zhang, Xingyuan Pan, Minrui Xu, Jipeng Zhang, Renjie Pi, Xiaoyu Wang, and Tong Zhang. Scalebio: Scalable bilevel optimization for llm data reweighting. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 31959–31982, 2025.
- Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.
- Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *International Conference on Learning Representations*, volume 2024, pp. 30988–31043, 2024.
- GuanWen Qiu, Da Kuang, and Surbhi Goel. Complexity matters: Feature learning in the presence of spurious correlations. In *Forty-first International Conference on Machine Learning*, 2024.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 Technical Report, January 2025. URL <http://arxiv.org/abs/2412.15115>. arXiv:2412.15115 [cs].

- Mengye Ren, Wenyuan Zeng, Bin Yang, and Raquel Urtasun. Learning to reweight examples for robust deep learning. In *International conference on machine learning*, pp. 4334–4343. PMLR, 2018.
- Luca Scimeca, Seong Joon Oh, Sanghyuk Chun, Michael Poli, and Sangdoo Yun. Which short-cut cues will dnns choose? a study from the parameter-space perspective. *arXiv preprint arXiv:2110.03095*, 2021.
- Harshay Shah, Kaustav Tamuly, Aditi Raghunathan, Prateek Jain, and Praneeth Netrapalli. The pitfalls of simplicity bias in neural networks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- Han Shen and Tianyi Chen. On penalty-based bilevel gradient descent method. In *International conference on machine learning*, pp. 30992–31015. PMLR, 2023.
- Han Shen, Pin-Yu Chen, Payel Das, and Tianyi Chen. Seal: Safety-enhanced aligned llm fine-tuning via bilevel data selection. In *International Conference on Learning Representations*, volume 2025, pp. 31243–31264, 2025a.
- Han Shen, Zhuoran Yang, and Tianyi Chen. Principled penalty-based methods for bilevel reinforcement learning and rlhf. *Journal of Machine Learning Research*, 26(114):1–49, 2025b.
- Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. *Advances in neural information processing systems*, 32, 2019.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Yulin Wang, Jiayi Guo, Shiji Song, and Gao Huang. Meta-semi: A meta-learning approach for semi-supervised learning. *arXiv preprint arXiv:2007.02394*, 2020.
- Maurice Weber, Daniel Y Fu, Quentin Anthony, Yonatan Oren, Shane Adams, Anton Alexandrov, Xiaozhong Lyu, Huu Nguyen, Xiaozhe Yao, Virginia Adams, et al. Redpajama: an open dataset for training large language models. *Advances in neural information processing systems*, 37: 116462–116492, 2024.
- Bingbing Wen, Sirajul Salekin, Feiyang Kang, Bill Howe, Lucy Lu Wang, Javier Movellan, and Manjot Bilkhu. MixAtlas: Uncertainty-aware data mixture optimization for multimodal llm mid-training, 2026.
- Steven Euijong Whang, Yuji Roh, Hwanjun Song, and Jae-Gil Lee. Data collection and quality challenges in deep learning: a data-centric ai perspective: Se whang et al. *The VLDB Journal*, 32(4):791–813, 2023.
- Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning, 2024. URL <https://arxiv.org/abs/2402.04333>.
- Quan Xiao, Songtao Lu, and Tianyi Chen. A generalized alternating method for bilevel learning under the polyak–lojasiewicz condition. *arXiv preprint arXiv:2306.02422*, 2023.
- Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems*, volume 36, 2023a. doi: 10.52202/075280-3059.
- Sang Michael Xie, Shibani Santurkar, Tengyu Ma, and Percy S. Liang. Data selection for language models via importance resampling. In *Advances in Neural Information Processing Systems*, volume 36, pp. 34201–34227, 2023b. doi: 10.52202/075280-1482.
- Wanyun Xie, Francesco Tonin, and Volkan Cevher. Chameleon: A flexible data-mixing framework for language model pretraining and finetuning, 2025.

- Yueqi Xie, Minghong Fang, Renjie Pi, and Neil Gong. Gradsafe: Detecting jailbreak prompts for llms via safety-critical gradient analysis. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pp. 507–518, 2024.
- Kailai Yang, Xiao Liu, Lei Ji, Hao Li, Xiao Liang, Zhiwei Liu, Yeyun Gong, Peng Cheng, and Mao Yang. Data mixing agent: Learning to re-weight domains for continual pre-training, 2026.
- Yang Yu, Kai Han, Hang Zhou, Yehui Tang, Kaiqi Huang, Yunhe Wang, and Dacheng Tao. Llm data selection and utilization via dynamic bi-level optimization. *arXiv preprint arXiv:2507.16178*, 2025.
- Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, and Xia Hu. Data-centric ai: Perspectives and challenges. *arXiv preprint arXiv:2301.04819*, 2023.
- Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025.
- Xuanhe Zhou, Junxuan He, Wei Zhou, Haodong Chen, Zirui Tang, Haoyu Zhao, Xin Tong, Guoliang Li, Youmin Chen, Jun Zhou, et al. A survey of LLM $\times$ DATA. *arXiv preprint arXiv:2505.18458*, 2025.

APPENDIX CONTENTS

A	Gradient of the ScaleBiO meta-network Objective.	16
B	Proof of Theorem 1	16
C	Proof of Theorem 2	19
D	Experiment Details	23
D.1	Datasets	23
D.2	Implementation Details	23
D.3	Input Features to the Meta-Network	23
D.4	Evaluation Details	24
D.5	KL Loss in Ablation Study	24
E	Toy Experiment in Sec.4.	24
F	Small-to-Large Model Generalization	25
G	Conclusion	25

A GRADIENT OF THE SCALEBIO META-NETWORK OBJECTIVE

Notation. Let $\mathcal{D}_{\text{train}} = \{x_i\}_{i=1}^N$, where $N = N_{\text{train}}$ is the number of training samples. Let $\ell(\theta; x_i)$ denote the token-averaged cross-entropy loss of an LLM parameterized by θ on sample x_i . For the selection network s_ϕ , define

$$w_i := s_\phi(x_i) > 0, \quad S := \sum_{j=1}^N w_j, \quad p_i := \frac{w_i}{S}.$$

The two LLMs are held fixed at θ_U^* and θ_W^* . Define the per-sample pseudo-labels and their weighted average as

$$\begin{aligned} \Delta_i^* &:= \ell(\theta_U^*; x_i) - \ell(\theta_W^*; x_i), \\ \bar{\Delta}^* &:= \sum_{j=1}^N p_j \Delta_j^* = \frac{\sum_{j=1}^N w_j \Delta_j^*}{S}. \end{aligned}$$

The pseudo-labels Δ_i^* are constant with respect to ϕ and the data weights, whereas $\bar{\Delta}^*$ depends on the weights through p_j .

meta-network objective. For the fixed LLMs, the objective in Eq. 6 becomes

$$\begin{aligned} \mathcal{L}_\phi^{\text{SBO}} &= \sum_{j=1}^N \frac{w_j}{S} [\ell(\theta_W^*; x_j) - \ell(\theta_U^*; x_j)] \\ &= - \sum_{j=1}^N p_j \Delta_j^* = - \frac{\sum_{j=1}^N w_j \Delta_j^*}{S} = -\bar{\Delta}^*. \end{aligned}$$

Gradient with respect to an individual weight. For a fixed index i , we differentiate with respect to w_i while holding all other weights fixed. Since the pseudo-labels are constant, the derivatives of the numerator and denominator are

$$\begin{aligned} \frac{\partial}{\partial w_i} \left(\sum_{j=1}^N w_j \Delta_j^* \right) &= \frac{\partial}{\partial w_i} \left(w_i \Delta_i^* + \sum_{j \neq i} w_j \Delta_j^* \right) = \Delta_i^*, \\ \frac{\partial S}{\partial w_i} &= \frac{\partial}{\partial w_i} \left(w_i + \sum_{j \neq i} w_j \right) = 1. \end{aligned}$$

Applying the quotient rule and substituting these derivatives gives

$$\begin{aligned} \frac{\partial \mathcal{L}_\phi^{\text{SBO}}}{\partial w_i} &= - \frac{S \frac{\partial}{\partial w_i} \left(\sum_{j=1}^N w_j \Delta_j^* \right) - \left(\sum_{j=1}^N w_j \Delta_j^* \right) \frac{\partial S}{\partial w_i}}{S^2} \\ &= - \frac{S \Delta_i^* - \sum_{j=1}^N w_j \Delta_j^*}{S^2} \\ &= - \frac{1}{S} \left(\Delta_i^* - \frac{\sum_{j=1}^N w_j \Delta_j^*}{S} \right) \\ &= - \frac{\Delta_i^* - \bar{\Delta}^*}{S}. \end{aligned} \tag{11}$$

B PROOF OF THEOREM 1

Notation and assumptions. Let $N = N_{\text{train}} \geq 2$ and fix the pseudo-labels $\Delta_i^* := \Delta_i(\theta_U^*, \theta_W^*)$. We treat the raw weights $\mathbf{w} = (w_1, \dots, w_N)$ as independent variables constrained only by $w_i \geq$

$\delta_0 \geq 0$, with $w_i(0) > \delta_0$ for every i . In particular, no upper bound is imposed on the weights. Assume there is a unique index K such that $\Delta_K^* > \Delta_i^*$ for all $i \neq K$.

For any feasible $\mathbf{w}$ with positive total weight, define

$$S(\mathbf{w}) = \sum_{j=1}^N w_j, \quad p_i(\mathbf{w}) = \frac{w_i}{S(\mathbf{w})}, \quad \bar{\Delta}^*(\mathbf{w}) = \sum_{i=1}^N p_i(\mathbf{w}) \Delta_i^*.$$

Writing the ScaleBiO objective as a function of the independent weights gives

$$\mathcal{L}^{\text{SBO}}(\mathbf{w}) = -\bar{\Delta}^*(\mathbf{w}), \quad \frac{\partial \mathcal{L}^{\text{SBO}}}{\partial w_i} = -\frac{\Delta_i^* - \bar{\Delta}^*(\mathbf{w})}{S(\mathbf{w})}.$$

Gradient descent and continuous-time dynamics. To establish the theorem, we first specify the weight updates and their continuous-time counterpart. Let n index discrete iterations and let $\eta_w > 0$ be the learning rate. Projected gradient descent under the constraint $w_i \geq \delta_0$ yields

$$w_i^{(n+1)} = \max \left\{ \delta_0, w_i^{(n)} + \frac{\eta_w}{S^{(n)}} (\Delta_i^* - \bar{\Delta}^{*(n)}) \right\}, \quad (12)$$

where $S^{(n)} = S(\mathbf{w}^{(n)})$ and $\bar{\Delta}^{*(n)} = \bar{\Delta}^*(\mathbf{w}^{(n)})$. The projection prevents an update from taking a weight below δ_0 .

The corresponding continuous-time dynamics, with the learning rate absorbed into time through $t = n\eta_w$, are described by the projected gradient flow

$$\dot{w}_i(t) = \begin{cases} \frac{\Delta_i^* - \bar{\Delta}^*(t)}{S(t)}, & w_i(t) > \delta_0, \\ \max \left\{ 0, \frac{\Delta_i^* - \bar{\Delta}^*(t)}{S(t)} \right\}, & w_i(t) = \delta_0, \end{cases} \quad (13)$$

where $S(t) = S(\mathbf{w}(t))$, $p_i(t) = p_i(\mathbf{w}(t))$, and $\bar{\Delta}^*(t) = \bar{\Delta}^*(\mathbf{w}(t))$. At the boundary, negative directions are suppressed while positive directions remain admissible. The proof below establishes the concentration result for this continuous-time flow.

Proof. Consider an absolutely continuous solution of Eq. 13. All differential identities and inequalities below hold almost everywhere, including across boundary transitions.

Define the pseudo-label range

$$R_\Delta := \Delta_K^* - \min_i \Delta_i^* > 0$$

and the set of coordinates following the unprojected direction:

$$\mathcal{I}(t) := \{i : w_i(t) > \delta_0 \text{ or } \Delta_i^* > \bar{\Delta}^*(t)\}.$$

Then

$$\dot{w}_i(t) = \begin{cases} (\Delta_i^* - \bar{\Delta}^*(t))/S(t), & i \in \mathcal{I}(t), \\ 0, & i \notin \mathcal{I}(t). \end{cases}$$

1. Positivity and growth of the normalization. Since the weights are nonnegative, $\bar{\Delta}^*(t)$ is a convex combination of the pseudo-labels. Thus $\bar{\Delta}^*(t) \leq \Delta_K^*$ and $\dot{w}_K(t) \geq 0$, which implies

$$S(t) \geq w_K(t) \geq w_K(0) > 0. \quad (14)$$

In particular, $w_K(t) > \delta_0$, so $K \in \mathcal{I}(t)$ for all t .

Moreover, $|\Delta_i^* - \bar{\Delta}^*(t)| \leq R_\Delta$. Hence $|\dot{w}_i(t)| \leq R_\Delta/w_K(0)$, excluding finite-time blow-up. Summing the coordinate derivatives gives

$$\begin{aligned} \frac{d}{dt} S(t)^2 &= 2S(t) \sum_i \dot{w}_i(t) \\ &= 2 \sum_{i \in \mathcal{I}(t)} (\Delta_i^* - \bar{\Delta}^*(t)) \leq 2NR_\Delta. \end{aligned} \quad (15)$$

Therefore,

$$S(t)^2 \leq S(0)^2 + 2NR_\Delta t. \quad (16)$$

By comparison with the reciprocals of this upper bound and its square root, respectively,

$$\int_0^\infty \frac{dt}{S(t)^2} = \infty, \quad \int_0^\infty \frac{dt}{S(t)} = \infty. \quad (17)$$

2. The weighted mean converges to the largest pseudo-label. Differentiating the weighted mean yields

$$\begin{aligned} \frac{d\bar{\Delta}^*(t)}{dt} &= \frac{1}{S(t)} \sum_i (\Delta_i^* - \bar{\Delta}^*(t)) \dot{w}_i(t) \\ &= \frac{1}{S(t)^2} \sum_{i \in \mathcal{I}(t)} (\Delta_i^* - \bar{\Delta}^*(t))^2 \geq 0. \end{aligned} \quad (18)$$

Thus $\bar{\Delta}^*(t)$ is nondecreasing and bounded above by Δ_K^* .

Let $V(t) = \Delta_K^* - \bar{\Delta}^*(t) \geq 0$. Since $K \in \mathcal{I}(t)$,

$$\dot{V}(t) \leq -\frac{V(t)^2}{S(t)^2}. \quad (19)$$

If V reaches zero, it remains zero by the monotonicity of $\bar{\Delta}^*$. Otherwise, integrating $\frac{d}{dt}(1/V(t)) \geq 1/S(t)^2$ gives

$$\frac{1}{V(t)} \geq \frac{1}{V(0)} + \int_0^t \frac{ds}{S(s)^2}.$$

The first integral in Eq. 17 diverges, so $V(t) \rightarrow 0$. Consequently,

$$\bar{\Delta}^*(t) \longrightarrow \Delta_K^*. \quad (20)$$

3. The normalized weights concentrate on the unique maximizer. For $i \neq K$, define

$$d_i := \Delta_K^* - \Delta_i^* > 0, \quad d_{\min} := \min_{i \neq K} d_i > 0.$$

Then

$$V(t) = \sum_{i \neq K} p_i(t) d_i \geq d_{\min}(1 - p_K(t)).$$

Since $V(t) \rightarrow 0$, we have $p_K(t) \rightarrow 1$. For every $i \neq K$, $0 \leq p_i(t) \leq 1 - p_K(t)$, and hence $p_i(t) \rightarrow 0$.

4. Every nonmaximal weight reaches the floor in finite time. Fix $i \neq K$. By Eq. 20, there exists a finite T_i such that

$$\bar{\Delta}^*(t) - \Delta_i^* \geq \frac{d_i}{2}, \quad t \geq T_i.$$

Whenever $w_i(t) > \delta_0$ after T_i ,

$$\dot{w}_i(t) \leq -\frac{d_i}{2S(t)}. \quad (21)$$

If w_i never reached the floor after T_i , integration would give

$$w_i(t) \leq w_i(T_i) - \frac{d_i}{2} \int_{T_i}^t \frac{ds}{S(s)}.$$

The second integral in Eq. 17 diverges, so the right-hand side would eventually fall below δ_0 , a contradiction. Thus w_i reaches the floor in finite time.

Once at the floor at or after T_i , its unprojected direction remains negative, so the projection keeps it there. Since there are finitely many nonmaximal coordinates, there exists a finite T_0 such that

$$w_i(t) = \delta_0 \quad \text{for all } i \neq K \text{ and } t \geq T_0. \quad (22)$$

For $\delta_0 = 0$, only $w_K(t)$ remains positive after T_0 . Therefore, $p_K(t) = 1$ and $p_i(t) = 0$ for every $i \neq K$ and $t \geq T_0$, proving finite-time convergence.

5. Convergence rate for a positive floor. Suppose $\delta_0 > 0$ and define

$$H := (N - 1)\delta_0 > 0, \quad C := \delta_0 \sum_{i \neq K} d_i > 0.$$

For $t \geq T_0$, Eq. 22 gives $S(t) = w_K(t) + H$ and

$$\Delta_K^* - \bar{\Delta}^*(t) = \frac{\delta_0 \sum_{i \neq K} d_i}{S(t)} = \frac{C}{S(t)}.$$

Only w_K changes after T_0 , so

$$\dot{S}(t) = \dot{w}_K(t) = \frac{C}{S(t)^2}.$$

Multiplying by $3S(t)^2$ and integrating yields

$$S(t)^3 = S(T_0)^3 + 3C(t - T_0). \quad (23)$$

Consequently,

$$\begin{aligned} 1 - p_K(t) &= \frac{H}{S(t)} \\ &= \frac{H}{[S(T_0)^3 + 3C(t - T_0)]^{1/3}} = O(t^{-1/3}). \end{aligned} \quad (24)$$

This completes the proof. $\square$

C PROOF OF THEOREM 2

Notation and comparison condition. Denote the empirical covariance between random variables u and v as $\widehat{\text{Cov}}_N(u, v) := N^{-1} \sum_{i=1}^N (u_i - \bar{u})(v_i - \bar{v})$. Further, denote each training data point as $\mathbf{x}_i = (1, e_i, h_i)^\top$. The features e_i and h_i are standardized to zero mean and unit standard deviation. The pseudo-labels $\Delta_i = \Delta_i(\theta_U^*, \theta_W^*)$ are fixed. For compactness, write $x_i^{(e)} = e_i$ and $x_i^{(h)} = h_i$, we have

$$\sum_i x_i^{(k)} = 0, \quad \frac{1}{N} \sum_i \left(x_i^{(k)}\right)^2 = 1, \quad k \in \{e, h\}. \quad (25)$$

We call e the easy feature and h the hard feature because e has stronger covariance with the target label than h ,

$$c_e = \widehat{\text{Cov}}_N(\Delta, e), \quad c_h = \widehat{\text{Cov}}_N(\Delta, h), \quad |c_e| > |c_h|. \quad (26)$$

As each feature is centered, for any vector $v \in \mathbb{R}^N$,

$$\begin{aligned} \widehat{\text{Cov}}_N(v, x^{(k)}) &= \frac{1}{N} \sum_i (v_i - \bar{v}) x_i^{(k)} \\ &= \frac{1}{N} \sum_i v_i x_i^{(k)} - \frac{\bar{v}}{N} \sum_i x_i^{(k)} \\ &= \frac{1}{N} \sum_i v_i x_i^{(k)}, \quad \bar{v} = \frac{1}{N} \sum_i v_i. \end{aligned} \quad (27)$$

Taking $v = \Delta^*$ gives

$$c_k = \widehat{\text{Cov}}_N(\Delta, x^{(k)}) = \frac{1}{N} \sum_i \Delta_i x_i^{(k)}, \quad \sum_i \Delta_i x_i^{(k)} = N c_k, \quad k \in \{e, h\}. \quad (28)$$

We have a meta-network parameterized by $\phi_t = (\beta_{0,t}, \beta_{e,t}, \beta_{h,t})$. For each training data point $\mathbf{x}_i$ and training step t , the output $w_{i,t}$ of the meta-network is computed in the following way.

$$a_{i,t} = \beta_t^\top \mathbf{x}_i = \beta_{0,t} + \beta_{e,t} e_i + \beta_{h,t} h_i, \quad w_{i,t} = s_{\phi_t}(x_i) = a_{i,t}. \quad (29)$$

Since the weights of data points are expected as non-negative, we assume that the initialization of parameters ϕ_0 satisfies $\beta_{e,0} = \beta_{h,0} = 0$ and $\beta_{0,0} > 0$ ensuring all $w_{i,0} > 0$. Then there exists a training step $T > 0$, such that $\forall t < T$, $w_{i,t} > 0$ for all i . We only consider the training steps $t < T$.

Recall the two objectives:

$$\mathcal{L}^{\text{SBO}}(\phi_t^{\text{SBO}}) = -\frac{\sum_i w_{i,t}^{\text{SBO}} \Delta_i}{S_t}, \quad \mathcal{L}^{\text{PVM}}(\phi_t^{\text{PVM}}) = \frac{1}{N} \sum_i (w_{i,t}^{\text{PVM}} - \Delta_i)^2, \quad (30)$$

where $S_t = \sum_i w_{i,t}^{\text{SBO}} > 0$ and $\bar{\Delta}_t = \sum_i w_{i,t}^{\text{SBO}} \Delta_i / S_t$. We write $G_k^q(\phi_t^q) = \partial \mathcal{L}^q(\phi) / \partial \beta_k |_{\phi=\phi_t^q}$ for $q \in \{\text{SBO}, \text{PVM}\}$ and $k \in \{e, h\}$.

Proof. We first compare the feature gradients of ScaleBiO.

1. ScaleBiO shows a non-decreasing gap between easy and hard features. We first derive the ScaleBiO gradient with respect to parameters. Since the pseudo-labels are fixed, the quotient rule gives

$$\begin{aligned} G_k^{\text{SBO}}(\phi_t^{\text{SBO}}) &= -\frac{1}{S_t} \sum_i \Delta_i \frac{\partial w_{i,t}^{\text{SBO}}}{\partial \beta_{k,t}^{\text{SBO}}} + \frac{\sum_j w_{j,t}^{\text{SBO}} \Delta_j^*}{S_t^2} \sum_i \frac{\partial w_{i,t}^{\text{SBO}}}{\partial \beta_{k,t}^{\text{SBO}}} \\ &= -\frac{1}{S_t} \sum_i (\Delta_i - \bar{\Delta}_t) \frac{\partial w_{i,t}^{\text{SBO}}}{\partial \beta_{k,t}^{\text{SBO}}}. \end{aligned} \quad (31)$$

The meta-network definition implies

$$\frac{\partial w_{i,t}^{\text{SBO}}}{\partial \beta_{k,t}^{\text{SBO}}} = x_i^{(k)}. \quad (32)$$

Substituting this into Eq. equation 31 yields

$$\begin{aligned} G_k^{\text{SBO}}(\phi_t^{\text{SBO}}) &= -\frac{1}{S_t} \sum_i (\Delta_i - \bar{\Delta}_t) x_i^{(k)} \\ &= -\frac{1}{S_t} \left[\sum_i \Delta_i x_i^{(k)} - \bar{\Delta}_t \sum_i x_i^{(k)} \right] \\ &= -\frac{1}{S_t} \sum_i \Delta_i x_i^{(k)} \\ &= -\frac{N}{S_t} c_k. \end{aligned} \quad (33)$$

The third equality uses $\sum_i x_i^{(k)} = 0$. The final equality uses $\sum_i \Delta_i x_i^{(k)} = N c_k$ from Eq. 28. Since the positive factor N/S_t is shared by both features,

$$(G_e^{\text{SBO}}(\phi_t^{\text{SBO}}))^2 - (G_h^{\text{SBO}}(\phi_t^{\text{SBO}}))^2 = \left(\frac{N}{S_t}\right)^2 (c_e^2 - c_h^2) > 0. \quad (34)$$

Next, we show that S_t is non-increasing,

$$\begin{aligned} S_t &= \sum_i w_{i,t}^{\text{SBO}} \\ &= N \beta_{0,t}^{\text{SBO}} + \beta_{e,t}^{\text{SBO}} \sum_i e_i + \beta_{h,t}^{\text{SBO}} \sum_i h_i \\ &= N \beta_{0,t}^{\text{SBO}}. \end{aligned} \quad (35)$$

Define

$$z_t = c_e \beta_{e,t}^{\text{SBO}} + c_h \beta_{h,t}^{\text{SBO}}. \quad (36)$$

Using Eq. equation 33, the gradient descent updates give

$$\beta_{k,t+1}^{\text{SBO}} = \beta_{k,t}^{\text{SBO}} + \eta \frac{N}{S_t} c_k, \quad k \in \{e, h\}. \quad (37)$$

Consequently,

$$z_{t+1} = z_t + \eta \frac{N}{S_t} (c_e^2 + c_h^2). \quad (38)$$

Since $z_0 = 0$ and $S_t > 0$, it follows that $z_t \geq 0$.

To determine the bias update, write $\Delta_{\text{avg}} = N^{-1} \sum_i \Delta_i$. Eq. 35 and Eq. 28 imply

$$\begin{aligned} \sum_i w_{i,t}^{\text{SBO}} \Delta_i &= \beta_{0,t}^{\text{SBO}} \sum_i \Delta_i + N z_t \\ &= S_t \Delta_{\text{avg}} + N z_t. \end{aligned} \quad (39)$$

Therefore,

$$\mathcal{L}^{\text{SBO}}(\phi_t^{\text{SBO}}) = -\Delta_{\text{avg}} - \frac{N z_t}{S_t}. \quad (40)$$

Since z_t does not depend on the bias coordinate and $\partial S_t / \partial \beta_t^{\text{SBO}} = N$, the bias gradient is

$$\frac{d\mathcal{L}^{\text{SBO}}}{d\beta_{0,t}} = \frac{N^2}{S_t^2} z_t \geq 0. \quad (41)$$

Thus the bias update satisfies

$$\beta_{0,t+1}^{\text{SBO}} = \beta_{0,t}^{\text{SBO}} - \eta \frac{N^2}{S_t^2} z_t \leq \beta_{0,t}^{\text{SBO}}. \quad (42)$$

Combining this with Eq. equation 35 yields

$$S_{t+1} = S_t - \eta \frac{3N^3}{S_t^2} z_t \leq S_t. \quad (43)$$

Finally, Eq. equation 34 gives

$$\begin{aligned} D_t^{\text{SBO}} &= \left(\frac{N}{S_t} \right)^2 (c_e^2 - c_h^2) \\ &= \left(\frac{N}{S_{t-1}} \right)^2 (c_e^2 - c_h^2) \\ &\geq D_{t-1}^{\text{SBO}}. \end{aligned} \quad (44)$$

Thus, ScaleBiO continues to learn easy features than hard features, and their gradient gap does not decrease. Next we show the exponentially decreasing of gradient gap with PVM.

2. PVM exhibits a exponentially decreasing gradient gap. We now analyze the PVM objective under the same fixed affine surrogate. Define the residual

$$r_{i,t} = w_{i,t}^{\text{PVM}} - \Delta_i. \quad (45)$$

Since the pseudo-labels are fixed, the feature gradients are

$$\begin{aligned} G_k^{\text{PVM}}(\phi_t^{\text{PVM}}) &= \frac{2}{N} \sum_i r_{i,t} \frac{\partial w_{i,t}^{\text{PVM}}}{\partial \beta_{k,t}^{\text{PVM}}} \\ &= \frac{2}{N} \sum_i r_{i,t} x_i^{(k)}, \quad k \in \{e, h\}. \end{aligned} \quad (46)$$

To include the bias update, write The full gradient is therefore

$$x_i = (1, e_i, h_i)^\top, \quad g_t = \nabla_\phi \mathcal{L}^{\text{PVM}}(\phi_t^{\text{PVM}}). \quad (47)$$

The full gradient is therefore

$$g_t = \frac{2}{N} \sum_i r_{i,t} x_i. \quad (48)$$

Each network output is,

$$w_{i,t}^{\text{PVM}} = x_i^\top \phi_t^{\text{PVM}}. \quad (49)$$

Combining this with the gradient descent update $\phi_{t+1}^{\text{PVM}} = \phi_t^{\text{PVM}} - \eta g_t$ gives

$$\begin{aligned} r_{i,t+1} &= r_{i,t} + x_i^\top (\phi_{t+1}^{\text{PVM}} - \phi_t^{\text{PVM}}) \\ &= r_{i,t} - \eta x_i^\top g_t. \end{aligned} \quad (50)$$

Substituting Eq. equation 50 into Eq. equation 48 yields

$$\begin{aligned} g_{t+1} &= \frac{2}{N} \sum_i (r_{i,t} - \eta x_i^\top g_t) x_i \\ &= g_t - 2\eta \left(\frac{1}{N} \sum_i x_i x_i^\top \right) g_t \\ &= (I - 2\eta Q) g_t, \end{aligned} \quad (51)$$

where

$$Q = \frac{1}{N} \sum_i x_i x_i^\top. \quad (52)$$

Because the features are centered and have unit variance,

$$Q = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & \rho \\ 0 & \rho & 1 \end{pmatrix}, \quad \rho = \frac{1}{N} \sum_i e_i h_i. \quad (53)$$

For brevity, write $G_{k,t}^{\text{PVM}} = G_k^{\text{PVM}}(\phi_t^{\text{PVM}})$. Equations equation 51 and equation 53 imply

$$\begin{aligned} G_{e,t+1}^{\text{PVM}} &= (1 - 2\eta) G_{e,t}^{\text{PVM}} - 2\eta \rho G_{h,t}^{\text{PVM}}, \\ G_{h,t+1}^{\text{PVM}} &= (1 - 2\eta) G_{h,t}^{\text{PVM}} - 2\eta \rho G_{e,t}^{\text{PVM}}. \end{aligned} \quad (54)$$

Consequently, the squared-gradient gap satisfies

$$\begin{aligned} D_{t+1}^{\text{PVM}} &= [(1 - 2\eta) G_{e,t}^{\text{PVM}} - 2\eta \rho G_{h,t}^{\text{PVM}}]^2 \\ &\quad - [(1 - 2\eta) G_{h,t}^{\text{PVM}} - 2\eta \rho G_{e,t}^{\text{PVM}}]^2 \\ &= [(1 - 2\eta)^2 - 4\eta^2 \rho^2] [(G_{e,t}^{\text{PVM}})^2 - (G_{h,t}^{\text{PVM}})^2] \\ &= [(1 - 2\eta)^2 - 4\eta^2 \rho^2] D_t^{\text{PVM}}. \end{aligned} \quad (55)$$

The cross terms cancel in the third equality.

Choose the step size such that:

$$0 < \eta < \frac{1}{2 + 2|\rho|}. \quad (56)$$

Then

$$0 < (1 - 2\eta)^2 - 4\eta^2 \rho^2 = [1 - 2\eta(1 + \rho)][1 - 2\eta(1 - \rho)] < 1. \quad (57)$$

Define

$$\lambda = 1 - [(1 - 2\eta)^2 - 4\eta^2 \rho^2] = 4\eta - (1 - \rho^2)4\eta^2, \quad \lambda \in (0, 1) \quad (58)$$

Equation equation 55 therefore becomes

$$D_{t+1}^{\text{PVM}} = (1 - \lambda) D_t^{\text{PVM}}. \quad (59)$$

Iterating this identity gives

$$D_t^{\text{PVM}} = (1 - \lambda)^t D_0^{\text{PVM}}. \quad (60)$$

Thus PVM exponentially contracts the absolute squared-gradient gap, while Eq. equation 44 shows that the ScaleBiO gap remains no lower than its initial value throughout the stated comparison interval. $\square$

D EXPERIMENT DETAILS

D.1 DATASETS

For LLM safety, we use Alpaca and Dolly as training datasets and evaluate on DirectHarm4, HarmBench, and HEx-PHI:

- Alpaca (Taori et al., 2023; Peng et al., 2023) contains 52,002 instruction–response pairs covering diverse instruction-following tasks.
- Dolly (Conover et al., 2023) contains 15,011 human-written instruction–response pairs covering tasks such as question answering, summarization, and creative writing.
- DirectHarm4 (DH4) (Lyu et al., 2024) contains 400 harmful instructions, with 100 examples in each of four categories: malware, drugs, phishing, and disinformation.
- HarmBench (HB) (Mazeika et al., 2024) evaluates harmful behavior generation. Its text-only portion contains 400 behaviors across standard, contextual, and copyright-related categories.
- HEx-PHI (HEx) (Qi et al., 2024) evaluates responses to instructions spanning prohibited-use categories. Its original release contains 330 instructions, with 30 examples in each of 11 categories: illegal activity, child abuse content, hate/harassment/violence, malware, physical harm, economic harm, fraud/deception, adult content, political campaigning, privacy violation, and tailored financial advice.

For targeted instruction tuning, we use Tulu V2 as the candidate instruction corpus and evaluate data selection for GSM8K and CodeX:

- Tulu V2 (Iverson et al., 2025) is a multi-source instruction-tuning corpus; we use the approximately 200K-example version preprocessed by Nayak et al. (2026), which excludes examples with no response tokens within the first 2,048 tokens. We train the meta-network on a random 50K subset and apply it to the full candidate pool.
- GSM8K (Cobbe et al., 2021) contains grade-school math problems with step-by-step solutions, comprising 7,473 training examples and 1,319 test examples. We hold out 1,000 examples from the training split as a shared validation set for all methods.
- CodeX (Chen et al., 2021) contains 164 Python function-completion problems with unit tests for evaluating functional correctness. Following Iverson et al. (2025), we use 16 examples as the query set and the remaining 148 as the test set.

D.2 IMPLEMENTATION DETAILS

meta-network learning. We train the two LLMs, θ_U and θ_W , using LISA (Pan et al., 2024), with learning rates in $\{5 \times 10^{-5}, 10^{-4}\}$ for up to three epochs. We tune the training–validation trade-off coefficient α over $\{0.01, 0.1, 1.0\}$. The meta-network is a three-layer MLP with a sigmoid output, optimized using AdamW with a learning rate of 10^{-3} .

Downstream fine-tuning for LLM safety. We apply LoRA to the query and key projections, W_q and W_k , in every Transformer layer, with rank $r = 8$ and $\alpha_{\text{LoRA}} = 8$. We train for three epochs with a batch size of 32, a learning rate of 10^{-4} , and a cosine learning rate schedule with a warmup ratio of 0.1.

Downstream fine-tuning for targeted instruction tuning. We fully finetune Llama-2-7B on the selected examples for two epochs using AdamW with a learning rate of 2×10^{-5} and a linear learning rate schedule. The maximum sequence length is 2,048 tokens.

D.3 INPUT FEATURES TO THE META-NETWORK

Inspired by Du et al. (2026), we construct sample-level features in both representation and gradient spaces. The representation embedding is the final-layer hidden state of the last token, and the gradient embedding is derived from the per-sample loss gradient. In each space, we compute

the distances to each of the three nearest validation examples and to the mean validation embedding. We concatenate these eight distances with the sample’s loss and gradient norm to obtain a 10-dimensional feature vector.

For LLM safety, we append a binary feature indicating whether the sample contains numbers or predefined keywords such as `list`, `convert`, and `calculate`. This feature is motivated by findings that some benign list-format and mathematical examples can degrade safety alignment after fine-tuning (Chen et al., 2024).

D.4 EVALUATION DETAILS

LLM safety. For DH4, HB, and HEx, we generate model responses using greedy decoding (temperature 0). We classify each response as safe or unsafe using Llama Guard, also with greedy decoding. We report the attack success rate (ASR), defined as the percentage of evaluated responses classified as unsafe; lower is better.

GSM8K. Following Iverson et al. (2025), we use the eight chain-of-thought demonstrations as individual query examples for data selection, without prepending other demonstrations to each query. At evaluation time, we include all eight demonstrations in each test prompt and report exact-match accuracy of the extracted final numerical answers on the 1,319 test problems.

CodeX. We sample code completions at a temperature of 0.8 and report pass@10 on the 148 held-out problems. This metric estimates the probability that at least one of ten sampled completions passes all unit tests for a problem.

D.5 KL LOSS IN ABLATION STUDY

The KL variant matches the normalized outputs of the meta-network $\mathbf{p}$ to a distribution $\mathbf{q}$ reweighted by the loss-gap pseudo-labels:

$$p_i = \frac{s_\phi(x_i)}{\sum_j s_\phi(x_j)}, \quad q_i = \frac{p_i \exp(\Delta_i^*)}{\sum_j p_j \exp(\Delta_j^*)}, \quad \mathcal{L}_\phi^{\text{KL}} = \text{KL}(\mathbf{q} \parallel \mathbf{p}).$$

E TOY EXPERIMENT IN SEC.4

We construct disjoint seen and unseen datasets, each containing 2,575 safe examples from GSM8K (Cobbe et al., 2021) and 1,125 unsafe examples from BeaverTails (Ji et al., 2023). The unsafe examples pair harmful prompts with harmful responses. For validation, we use 100 safety-related examples from He et al. (2024), each pairing a harmful prompt with a refusal response.

We train the meta-network on the seen set for 30 epochs using AdamW with a batch size of 256, an initial learning rate of 5×10^{-4} , and a cosine learning rate schedule that decays to zero.

For each split separately, we rank examples by predicted weight and classify the $k = 1,125$ lowest-weight examples as unsafe. Let $\mathcal{B}_k$ denote this predicted set and $\mathcal{U}$ the ground-truth unsafe set. We define unsafe-identification accuracy as

$$\text{Acc} = \frac{|\mathcal{B}_k \cap \mathcal{U}|}{k} \times 100\%. \quad (61)$$

Since $|\mathcal{U}| = k$, this metric equals both precision and recall for unsafe-example identification.

Shortcut learning. To test shortcut reliance, we append three synthetic features to each example before standardization. Let $c = +1$ for GSM8K examples and $c = -1$ for unsafe examples, and define $\mathbf{v} = (1, -0.5, 0.2)^\top$. We generate the shortcut features as

$$\mathbf{z}_{\text{seen}} = ac\mathbf{v} + \boldsymbol{\epsilon}, \quad \mathbf{z}_{\text{unseen}} = -ac\mathbf{v} + \boldsymbol{\epsilon}', \quad (62)$$

where a controls shortcut strength and $\boldsymbol{\epsilon}, \boldsymbol{\epsilon}' \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_3)$, with noise standard deviation $\sigma = 0.3$. We use $a = 0.5$ and $a = 0.3$ for the strong- and weak-shortcut settings, respectively. Noise is sampled independently across examples and coordinates. This construction reverses the class-dependent shortcut means on unseen data.

Table 5: Small-to-large model transfer with TESS. Small2Big uses Qwen2.5-0.5B-Instruct for meta-network training and Qwen2.5-7B-Instruct for fine-tuning on the selected data; Big2Big uses Qwen2.5-7B-Instruct for both. We report ASR (%) and training speedup S_{rel} relative to Big2Big within each dataset and setting. Avg. averages the six ASR scores, and Retention (%) is the ratio of Small2Big’s average ASR to Big2Big’s within the same setting. Higher is better for all metrics; better ASR and speedup results are **bold**.

Setting	Method	Alpaca				Dolly				Overall ASR	
		DH4	HB	HEX	S_{rel}	DH4	HB	HEX	S_{rel}	Avg.	Retention
Training	Big2Big	44.50	23.50	24.83	1.00×	86.50	87.00	88.62	1.00×	59.16	–
	Small2Big	35.50	13.00	13.79	6.60 ×	74.00	72.00	66.90	5.65 ×	45.87	77.53
Generalization	Big2Big	38.75	16.00	18.28	1.00×	84.50	83.50	80.00	1.00×	53.51	–
	Small2Big	32.25	13.00	14.14	6.20 ×	77.50	70.00	61.72	5.65 ×	44.77	83.67

For each split, we summarize the associations between the shortcut features and class labels before standardization. For feature $j \in \{1, 2, 3\}$, let

$$\hat{C}_j = \frac{1}{n-1} \sum_{i=1}^n (z_{ij} - \bar{z}_j)(c_i - \bar{c}) \quad (63)$$

denote its empirical covariance with the label, where n is the split size and $\bar{z}_j$ and $\bar{c}$ are the sample means. We report the mean absolute covariance and mean absolute Pearson correlation:

$$\text{Cov} = \frac{1}{3} \sum_{j=1}^3 |\hat{C}_j|, \quad \rho = \frac{1}{3} \sum_{j=1}^3 \left| \frac{\hat{C}_j}{\hat{\sigma}_j \hat{\sigma}_c} \right|, \quad (64)$$

where $\hat{\sigma}_j$ and $\hat{\sigma}_c$ are the sample standard deviations of feature j and the label, respectively.

We report seen accuracy Acc_S , unseen accuracy Acc_U , and their gap $G = \text{Acc}_S - \text{Acc}_U$ in percentage points. At comparable seen accuracy, a smaller gap indicates better generalization under shortcut reversal.

F SMALL-TO-LARGE MODEL GENERALIZATION

Table 5 provides the detailed results of the small-to-large model transfer experiment summarized in Sec. 5.1. Small2Big achieves a 5.65–6.60× training speedup while retaining 77.53% and 83.67% of Big2Big’s average ASR in the *Training* and *Generalization* settings, respectively. These results show that a meta-network trained with TESS using a small LLM can generalize to data selection for a larger model. performance.

G CONCLUSION

In this paper, we introduced Transferable Example Scoring and Selection (TESS), transferable data-selection framework that utilize sample-level pseudo-labels to learn a data selection network generalizable to unseen data. Our theoretical and empirical analyses identify competitive weight suppression and persistent easy-feature preference as limitations of the ScaleBiO meta-objective. To address these issues, we propose a new training objective PVM. Experiments demonstrate cross-dataset, subset-to-full-corpus, and small-to-large model transfer. Specifically, in terms of generalization, TESS outperforms the strongest learnable-weighting baselines by 22.75 and 3.99 points on LLM safety and targeted instruction tuning, respectively.